

DOCUMENTACIÓN TÉCNICA

Notebook: [eda_heart_failure_R.ipynb](#)

Guía Paso a Paso — EDA, Preprocesamiento y Entrenamiento

Heart Failure Prediction Dataset · XGBoost + Optuna + SMOTE

Madrid, marzo de 2026

Índice de contenidos

- FASE 1** Importación de librerías
- FASE 2** Carga del dataset
- FASE 3** Exploración inicial (EDA)
- FASE 4** Limpieza y preprocesamiento
- FASE 5** Análisis exploratorio visual
- FASE 6** Codificación One-Hot Encoding
- FASE 7** Diccionario de variables
- FASE 8** Análisis de correlación (Heatmap)
- FASE 9** Análisis bivariado
- FASE 10** Pairplot multivariado
- FASE 11** Separación X / y y división Train-Test
- FASE 12** Modelo base XGBoost
- FASE 13** Optimización con Optuna (F1)
- FASE 14** Regularización manual y control overfitting
- FASE 15** Validación cruzada estratificada
- FASE 16** Balanceo con SMOTE
- FASE 17** Análisis de umbrales de decisión
- FASE 18** Segunda optimización con Optuna (Recall)
- FASE 19** Modelo Maestro Final
- FASE 20** Evaluación final y exportación

FASE 1 — Importación de Librerías

El primer paso de cualquier proyecto de Machine Learning consiste en importar todas las herramientas que se utilizarán a lo largo del notebook. En Python, las librerías son colecciones de funciones y clases que extienden las capacidades del lenguaje. Agruparlas al inicio facilita la lectura del código y permite identificar de un vistazo todas las dependencias del proyecto.

Grupos de librerías importadas

Manipulación y análisis de datos: pandas (pd) para trabajar con tablas de datos (DataFrames) y numpy (np) para operaciones matemáticas y manejo de valores nulos (NaN).

Visualización: matplotlib.pyplot (plt) para crear figuras y gráficos base, y seaborn (sns) para visualizaciones estadísticas avanzadas como heatmaps y boxplots.

Preprocesamiento: LabelEncoder de sklearn para convertir texto a números y SimpleImputer para rellenar valores faltantes con la mediana.

División y validación: train_test_split para separar los datos, StratifiedKFold y cross_val_score para validación cruzada, y learning_curve para detectar sobreajuste.

Balanceo de clases: SMOTE de imbalanced-learn para generar datos sintéticos y equilibrar las clases.

Métricas: classification_report, confusion_matrix, ConfusionMatrixDisplay, roc_curve, auc y RocCurveDisplay para evaluar el rendimiento del modelo.

Modelo: XGBClassifier y plot_importance de xgboost.

Optimización: optuna para la búsqueda bayesiana de hiperparámetros.

Guardado: joblib para serializar y guardar el modelo entrenado en disco.

Código

```
Python
import pandas as pd                # Manipulación y limpieza de datos
(DataFrames).
import numpy as np                # Operaciones matemáticas y manejo de
NaN.
import matplotlib.pyplot as plt    # Creación de gráficos base.
import seaborn as sns             # Visualización estadística avanzada.
from sklearn.preprocessing import LabelEncoder    # Codificación texto →
número.
from sklearn.impute import SimpleImputer          # Relleno de valores nulos.
from sklearn.model_selection import train_test_split    # División Train / Test.
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.model_selection import learning_curve
from imblearn.over_sampling import SMOTE           # Balanceo de clases.
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import roc_curve, auc, RocCurveDisplay
from xgboost import XGBClassifier, plot_importance
import optuna
import joblib
```

FASE 2 — Carga del Dataset

El archivo `heart.csv` contiene los registros clínicos de 918 pacientes. Se carga mediante `pd.read_csv()` y se asigna a la variable `data`, que será la base de datos principal durante todo el proyecto.

Código

```
Python
data = pd.read_csv('../data/heart.csv')
```

La ruta './data/heart.csv' utiliza dos puntos (..) para subir un nivel desde la carpeta actual y acceder a la carpeta data donde está almacenado el archivo.

FASE 3 — Exploración Inicial del Dataset (EDA)

Antes de transformar cualquier dato, es imprescindible entender qué contiene el dataset: cuántas filas y columnas tiene, qué tipo de datos hay en cada columna, si existen valores nulos y cuál es el rango estadístico de las variables numéricas. Esta fase corresponde al Análisis Exploratorio de Datos (EDA).

Paso 3.1 — Dimensiones del dataset

`data.shape` devuelve una tupla (filas, columnas) que indica el volumen total de información disponible.

```
Python
data.shape
```

Output:
(918, 12)

El dataset tiene 918 pacientes y 12 columnas (variables clínicas).

Paso 3.2 — Primeras filas

`data.head(5)` muestra los primeros 5 registros para verificar que la carga fue correcta y entender el formato de los datos.

```
Python
data.head(5)
```

Output:	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR
0	40	M	ATA	140	289	0	Normal	172
1	49	F	NAP	160	180	0	Normal	156
2	37	M	ATA	130	283	0	ST	98

Paso 3.3 — Estadísticas descriptivas

`data.describe()` calcula automáticamente estadísticas para todas las columnas numéricas: conteo, media, desviación estándar, mínimo, percentiles (25%, 50%, 75%) y máximo. Esta visión global permite detectar anomalías como valores cero en colesterol o presión arterial.

```
Python
data.describe()
```

Output:

	Age	RestingBP	Cholesterol	FastingBS	MaxHR
count	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368
std	9.432617	18.514154	109.384145	0.423046	25.460334
min	28.000000	0.000000	0.000000	0.000000	60.000000
max	77.000000	200.000000	603.000000	1.000000	202.000000

¡Alerta! Se detectan valores mínimos de 0 en Cholesterol y RestingBP, lo que es fisiológicamente imposible. Estos se corregirán en la Fase 4.

Paso 3.4 — Columnas numéricas

`data.select_dtypes(include='number').columns` lista exclusivamente las columnas con tipo de dato numérico, separándolas de las categóricas (texto).

```
Python
data.select_dtypes(include='number').columns
```

Output:

```
Index(['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak',  
'HeartDisease'], dtype='str')
```

Paso 3.5 — Información general (data.info)

`data.info()` proporciona una radiografía completa del DataFrame: número total de registros, nombre de columnas, valores no nulos por columna, tipo de dato (Dtype) y uso de memoria RAM.

```
Python
data.info()
```

Output:

```
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                   918 non-null   int64
1   Sex                   918 non-null   str
2   ChestPainType         918 non-null   str
3   RestingBP             918 non-null   float64
4   Cholesterol           918 non-null   float64
5   FastingBS             918 non-null   int64
6   RestingECG            918 non-null   str
7   MaxHR                 918 non-null   int64
8   ExerciseAngina        918 non-null   str
9   Oldpeak               918 non-null   float64
10  ST_Slope              918 non-null   str
11  HeartDisease          918 non-null   int64
```

Paso 3.6 — Conteo de valores nulos

`data.isnull().sum()` marca cada celda como True si es NaN y suma por columna. El resultado confirma cuántos valores faltan en cada variable antes de la limpieza.

```
Python
data.isnull().sum()
```

Output:	
Age	0
Sex	0
ChestPainType	0
RestingBP	0
Cholesterol	0
FastingBS	0
RestingECG	0
MaxHR	0
ExerciseAngina	0
Oldpeak	0
ST_Slope	0
HeartDisease	0

No hay valores NaN explícitos, pero como se vio en `describe()`, los ceros en Cholesterol y RestingBP son valores incorrectos que deben tratarse como datos faltantes.

FASE 4 — Limpieza y Preprocesamiento de Datos

Paso 4.1 — Imputación de valores fisiológicamente imposibles

Un nivel de colesterol de 0 mg/dl o una presión arterial de 0 mmHg son biológicamente inviables. Estos valores representan errores de registro, no datos reales. El proceso es:

- Reemplazar los ceros por NaN (valor nulo formal de NumPy) usando `.replace(0, np.nan)`.
- Instanciar `SimpleImputer` con `strategy='median'`, que calcula la mediana de cada columna ignorando los NaN.
- Aplicar `fit_transform()` para calcular la mediana y rellenar los NaN en una sola operación.

Se usa la mediana en lugar de la media porque es más robusta frente a valores extremos (outliers): no se ve distorsionada por colesteroles muy altos o muy bajos.

```
Python
data['Cholesterol'] = data['Cholesterol'].replace(0, np.nan)
data['RestingBP'] = data['RestingBP'].replace(0, np.nan)

imputer = SimpleImputer(strategy='median')
data[['Cholesterol', 'RestingBP']] = imputer.fit_transform(data[['Cholesterol',
'RestingBP']])
```

Paso 4.2 — Label Encoding de variables binarias

Las columnas Sex ('M'/'F') y ExerciseAngina ('Y'/'N') contienen solo dos categorías. LabelEncoder las convierte en 0 y 1 de forma automática. No se usa One-Hot Encoding aquí porque son variables binarias (solo dos opciones), y un solo número basta para representarlas sin crear ambigüedad matemática.

```
Python
le = LabelEncoder()
data['Sex'] = le.fit_transform(data['Sex'])
data['ExerciseAngina'] = le.fit_transform(data['ExerciseAngina'])
```

Resultado: Sex: F=0, M=1 | ExerciseAngina: N=0, Y=1.

Paso 4.3 — Conversión de booleanos a enteros

Algunas operaciones de Pandas (como get_dummies()) pueden generar columnas de tipo bool (True/False). XGBoost y Scikit-Learn trabajan mejor con enteros (1/0). Este paso convierte automáticamente todas las columnas booleanas del DataFrame.

```
Python
data = data.astype({col: int for col in data.select_dtypes('bool').columns})
```

El diccionario de comprensión {col: int for col in ...} genera automáticamente un mapeo de cada columna bool al tipo int, y astype() aplica la conversión de una sola vez.

Paso 4.4 — Verificación post-limpieza

Tras la imputación, se verifica que los ceros hayan desaparecido comprobando la frecuencia de valores en Cholesterol y RestingBP con value_counts(). También se analiza el balance de clases en la variable objetivo.

```
Python
data['Cholesterol'].value_counts()
data['RestingBP'].value_counts()
data['HeartDisease'].value_counts(normalize=True)
```

Output:

```
Cholesterol: 237.0 → 178 registros (valor más frecuente = mediana imputada)
RestingBP: 120.0 → 132 registros
```

```
HeartDisease:
1    0.553377 (55.3% enfermos)
0    0.446623 (44.6% sanos)
```

El dataset está ligeramente desbalanceado a favor de los enfermos, lo cual será tratado con SMOTE en fases posteriores.

FASE 5 — Análisis Exploratorio Visual

La visualización de datos permite detectar patrones, distribuciones anómalas y relaciones entre variables que los números solos no revelan. Se generan cuatro tipos de gráficos en esta fase.

Paso 5.1 — Histogramas con KDE

Se generan histogramas para las 6 variables numéricas principales (Age, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak). Cada histograma incluye una curva KDE (Kernel Density Estimation) que suaviza la distribución para identificar su forma (normal, sesgada, bimodal).

```
Python
num_cols = ['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak']
plt.figure(figsize=(12, 8))
for i, col in enumerate(num_cols, 1):
    plt.subplot(2, 3, i)
    sns.histplot(data[col], kde=True)
    plt.title(col)
plt.tight_layout()
plt.show()
```

Paso 5.2 — Boxplots para detectar outliers

Los diagramas de caja (boxplots) muestran la mediana, el rango intercuartílico (IQR) y los valores atípicos (puntos fuera de los bigotes). Son fundamentales para identificar registros extremos en variables como Cholesterol u Oldpeak.

```
Python
plt.figure(figsize=(12, 8))
for i, col in enumerate(num_cols, 1):
    plt.subplot(2, 3, i)
    sns.boxplot(y=data[col])
    plt.title(col)
plt.tight_layout()
plt.show()
```

Paso 5.3 — Distribución de la variable objetivo

Un gráfico de barras (countplot) muestra visualmente la proporción entre pacientes sanos (0) y enfermos (1), confirmando el ligero desbalance identificado en value_counts().

```
Python
plt.figure(figsize=(6,4))
sns.countplot(x='HeartDisease', data=data)
plt.title("Distribución de HeartDisease")
plt.show()
```

FASE 6 — Codificación One-Hot Encoding

Las columnas ChestPainType, RestingECG y ST_Slope contienen múltiples categorías sin orden jerárquico entre ellas. Si se usara Label Encoding (0, 1, 2...), el modelo interpretaría erróneamente que ASY > ATA > NAP en términos matemáticos, lo cual no es correcto.

La solución es One-Hot Encoding: cada categoría se convierte en una columna binaria independiente (0 o 1). pd.get_dummies() hace esto automáticamente y elimina las columnas originales.

```
Python
data = pd.get_dummies(data, columns=['ChestPainType', 'RestingECG', 'ST_Slope'])
```


Resultado: las 3 columnas originales se reemplazan por 9 columnas binarias nuevas:

- ChestPainType_ASY, ChestPainType_ATA, ChestPainType_NAP, ChestPainType_TA
- RestingECG_LVH, RestingECG_Normal, RestingECG_ST
- ST_Slope_Down, ST_Slope_Flat, ST_Slope_Up

Nota importante: ChestPainType_ASY (asintomático) es clínicamente el más peligroso: los pacientes sin dolor no buscan atención médica, lo que retrasa el diagnóstico de infarto silencioso.

FASE 7 — Diccionario de Variables Post-Procesamiento

Tras todas las transformaciones, el dataset cuenta con 19 columnas. La siguiente tabla documenta el significado de cada variable:

Variable	Significado clínico
Age	Edad del paciente en años.
Sex	Sexo (0: Mujer, 1: Hombre).
RestingBP	Presión arterial en reposo (mmHg). Imputada si era 0.
Cholesterol	Colesterol sérico (mg/dl). Imputado si era 0.
FastingBS	Azúcar en ayunas (1: > 120 mg/dl, 0: normal).
MaxHR	Frecuencia cardíaca máxima en ejercicio.
ExerciseAngina	Angina inducida por ejercicio (0: No, 1: Sí).
Oldpeak	Depresión del segmento ST en ejercicio vs. reposo.
HeartDisease	Variable objetivo (0: Sano, 1: Enfermo).
ChestPainType_ASY	Dolor torácico asintomático — el más peligroso (infarto silencioso).
ChestPainType_ATA	Angina atípica — síntomas no clásicos de angina.
ChestPainType_NAP	Dolor no anginoso — origen no cardíaco.
ChestPainType_TA	Angina típica — dolor clásico de opresión.
ST_Slope_Down	Pendiente ST descendente — asociada a isquemia activa.
ST_Slope_Flat	Pendiente ST plana — riesgo cardíaco intermedio-alto.
ST_Slope_Up	Pendiente ST ascendente — considerada normal.
RestingECG_LVH	ECG en reposo: hipertrofia ventricular izquierda.
RestingECG_Normal	ECG en reposo: sin anomalías.
RestingECG_ST	ECG en reposo: anomalía en onda ST-T (señal de isquemia).

FASE 8 — Matriz de Correlación (Heatmap)

La correlación de Pearson mide la fuerza y dirección de la relación lineal entre pares de variables (rango de -1 a +1). Un heatmap visualiza esta matriz: el rojo indica correlación positiva fuerte, el azul correlación negativa fuerte.

Este análisis es clave para entender qué variables tienen mayor relación con HeartDisease y para detectar colinealidad entre predictores.

```
Python
plt.figure(figsize=(10, 8))
corr = data.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
plt.title("Matriz de correlación")
plt.show()
```

Parámetros clave: `annot=True` inserta el coeficiente numérico en cada celda; `fmt='.2f'` lo redondea a 2 decimales; `cmap='coolwarm'` aplica la escala de colores divergente.

FASE 9 — Análisis Bivariado

El análisis bivariado compara la distribución de una variable numérica segmentada por la variable objetivo (Sano vs Enfermo). Se usan boxplots y gráficos de barras agrupados.

Paso 9.1 — Edad por enfermedad

```
Python
plt.figure(figsize=(6,4))
sns.boxplot(x='HeartDisease', y='Age', data=data)
plt.title("Edad por presencia de enfermedad")
plt.show()
```

Muestra si los pacientes enfermos tienden a ser de mayor edad. En cardiología, la edad es uno de los factores de riesgo más conocidos.

Paso 9.2 — Colesterol por enfermedad

```
Python
plt.figure(figsize=(6,4))
sns.boxplot(x='HeartDisease', y='Cholesterol', data=data)
plt.title("Colesterol por presencia de enfermedad")
plt.show()
```

Paso 9.3 — Frecuencia cardíaca máxima por enfermedad

```
Python
plt.figure(figsize=(6,4))
sns.boxplot(x='HeartDisease', y='MaxHR', data=data)
plt.title("MaxHR por presencia de enfermedad")
plt.show()
```

Los pacientes enfermos suelen mostrar una frecuencia cardíaca máxima menor, lo que puede indicar menor capacidad funcional del corazón.

Paso 9.4 — Tipo de dolor torácico por enfermedad

```
Python
chest_cols = ['ChestPainType_ASY', 'ChestPainType_ATA', 'ChestPainType_NAP',
              'ChestPainType_TA']
data[chest_cols +
      ['HeartDisease']].groupby('HeartDisease').sum().T.plot(kind='bar', figsize=(8,4))
plt.title("Tipo de dolor torácico por enfermedad")
plt.show()
```

`groupby('HeartDisease').sum()` agrupa por clase y suma las ocurrencias de cada tipo de dolor. La transposición (`.T`) sitúa los tipos en el eje X para facilitar la comparativa.

Paso 9.5 — Pendiente ST por enfermedad

```
Python
st_cols = ['ST_Slope_Down', 'ST_Slope_Flat', 'ST_Slope_Up']
data[st_cols + ['HeartDisease']].groupby('HeartDisease').sum().T.plot(kind='bar',
figsize=(8,4))
plt.title("Pendiente ST por enfermedad")
plt.show()
```

FASE 10 — Pairplot Multivariado

El pairplot genera una cuadrícula de gráficos que cruza todas las variables numéricas entre sí, segmentadas por color según el diagnóstico (`hue='HeartDisease'`). Es la visualización más completa del EDA:

- Diagonal: distribución KDE individual de cada variable para sanos y enfermos.
- Fuera de la diagonal: scatter plots que muestran la relación entre dos variables y la separabilidad entre clases.

```
Python
sns.pairplot(data[num_cols + ['HeartDisease']], hue='HeartDisease')
plt.show()
```

Al ser una operación costosa computacionalmente (genera 42 subgráficos para 6 variables), se ejecuta una sola vez al final del EDA.

FASE 11 — Separación X / y y División Train-Test

Paso 11.1 — Definición de X e y

Se separan las variables predictoras (X) de la variable objetivo (y). X contiene todas las columnas excepto `HeartDisease`. y contiene únicamente la columna `HeartDisease`.

Esta separación es obligatoria: el modelo nunca debe ver la respuesta correcta (y) durante el entrenamiento como si fuera una característica de entrada.

```
Python
X = data.drop('HeartDisease', axis=1)    # Todas las columnas menos la objetivo.
y = data['HeartDisease']                 # Solo la variable a predecir.
```

Paso 11.2 — División Train (80%) / Test (20%)

`train_test_split` divide X e y en cuatro conjuntos: `X_train` y `y_train` para entrenar el modelo, `X_test` e `y_test` para evaluarlo con datos que el modelo nunca ha visto.

```
Python
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,          # 20% para prueba → 184 pacientes.
    random_state=42         # Semilla fija para reproducibilidad.
)
```

Output:

`X_train: 734 pacientes | X_test: 184 pacientes`

`random_state=42` garantiza que siempre se genere la misma partición, haciendo los experimentos reproducibles y comparables.

FASE 12 — Modelo Base XGBoost

XGBoost (eXtreme Gradient Boosting) es un algoritmo de ensamble basado en árboles de decisión secuenciales. Cada árbol aprende de los errores del anterior (Gradient Boosting), lo que lo convierte en uno de los modelos más potentes para datos tabulares.

Como punto de partida, se entrena un modelo con la configuración por defecto para establecer una línea base de rendimiento.

Paso 12.1 — Entrenamiento del modelo base

```
Python
model = XGBClassifier(random_state=42)
model.fit(X_train, y_train)
```

Paso 12.2 — Evaluación del modelo base

```
Python
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

Output:

	precision	recall	f1-score	support
0	0.81	0.87	0.84	77
1	0.90	0.85	0.88	107

El modelo base alcanza un 86% de accuracy y un Recall de 0.85 para enfermos. El objetivo será mejorar este Recall mediante optimización.

Paso 12.3 — Matriz de confusión

```
Python
cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm, display_labels=['Sano', 'Enfermo']).plot(cmap='Blues')
plt.title("Matriz de confusión")
plt.show()
```

La matriz de confusión desglosa las predicciones en 4 cuadrantes: Verdaderos Negativos (sanos correctamente identificados), Falsos Positivos (sanos clasificados como enfermos), Falsos Negativos (enfermos no detectados — los más peligrosos) y Verdaderos Positivos (enfermos correctamente detectados).

Paso 12.4 — Curva ROC

```
Python
RocCurveDisplay.from_estimator(model, X_test, y_test)
plt.title("Curva ROC")
plt.show()
```

La curva ROC grafica la Tasa de Verdaderos Positivos (Recall) frente a la Tasa de Falsos Positivos para distintos umbrales de decisión. El AUC (Área Bajo la Curva) indica la capacidad global del modelo: 1.0 es perfecto, 0.5 equivale a azar.

Paso 12.5 — Importancia de variables

```
Python
plot_importance(model, max_num_features=10)
plt.title("Variables más importantes")
plt.show()
```

Paso 12.6 — Curvas de aprendizaje

```
Python
train_sizes, train_scores, val_scores = learning_curve(model, X, y, cv=5,
scoring='accuracy')
plt.plot(train_sizes, train_scores.mean(axis=1), label='Train')
plt.plot(train_sizes, val_scores.mean(axis=1), label='Validación')
plt.title("Curvas de aprendizaje")
plt.legend()
plt.show()
```

Las curvas de aprendizaje diagnostican el sobreajuste: si la línea de Train está muy por encima de la de Validación, el modelo memoriza en lugar de generalizar.

FASE 13 — Optimización de Hiperparámetros con Optuna (F1-Score)

Los hiperparámetros son configuraciones del modelo que no se aprenden durante el entrenamiento sino que se establecen previamente. Encontrar la combinación óptima manualmente sería inviable dado el espacio de búsqueda. Optuna automatiza este proceso mediante búsqueda bayesiana: aprende de cada evaluación para sugerir combinaciones más prometedoras.

Paso 13.1 — Definición de la función objetivo

La función `objective()` define qué hiperparámetros explorar y cómo evaluarlos. Optuna la llama repetidamente, cada vez con valores diferentes sugeridos por el algoritmo TPE (Tree-structured Parzen Estimator).

```
Python
def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3),
        'subsample': trial.suggest_float('subsample', 0.5, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'reg_alpha': trial.suggest_float('reg_alpha', 0, 10),
        'reg_lambda': trial.suggest_float('reg_lambda', 0, 10),
        'random_state': 42
    }
    model = XGBClassifier(**params)
    score = cross_val_score(model, X_train, y_train, cv=5, scoring='f1').mean()
    return score
```

Hiperparámetros explorados: `n_estimators` (número de árboles), `max_depth` (profundidad máxima), `learning_rate` (velocidad de aprendizaje), `subsample` (fracción de muestras por árbol), `colsample_bytree` (fracción de columnas por árbol), `reg_alpha` (regularización L1) y `reg_lambda` (regularización L2).

Paso 13.2 — Ejecución con Early Stopping

```
Python
study = optuna.create_study(direction='maximize')
study.optimize(
    objective,
    n_trials=200,
    callbacks=[lambda study, trial: study.stop()
               if study.best_trial.number + 20 < trial.number else None]
)
```

`direction='maximize'` indica que Optuna debe buscar la combinación que maximice el F1-Score. El callback de Early Stopping detiene la búsqueda si tras 20 intentos consecutivos no hay mejora, ahorrando tiempo de cómputo.

Paso 13.3 — Entrenamiento con mejores hiperparámetros

```
Python
best_model = XGBClassifier(**study.best_params, random_state=42)
best_model.fit(X_train, y_train)
y_pred_best = best_model.predict(X_test)
```

```
print(classification_report(y_test, y_pred_best, target_names=['Sano', 'Enfermo']))
```

Output:

	precision	recall	f1-score	support
Sano	0.81	0.88	0.84	77
Enfermo	0.91	0.85	0.88	107
accuracy			0.86	184

Paso 13.4 — Análisis de sobreajuste inicial

```
Python
train_score = best_model.score(X_train, y_train)
test_score = best_model.score(X_test, y_test)
print(f"Train: {train_score:.4f}")
print(f"Test: {test_score:.4f}")
print(f"Diferencia: {abs(train_score - test_score):.4f}")
```

Output:

```
Train: 0.9005
Test: 0.8641
Diferencia: 0.0364
```

Una diferencia de 0.0364 indica un leve sobreajuste. El siguiente paso aplica regularización adicional para reducirlo.

FASE 14 — Regularización Manual para Reducir Sobreajuste

Además de los parámetros encontrados por Optuna, se añaden restricciones manuales adicionales para controlar la complejidad del modelo:

- `max_depth = min(valor_optuna, 5)`: limita la profundidad máxima a 5 niveles para evitar árboles demasiado complejos.
- `min_child_weight = 5`: exige que un nodo tenga al menos el peso de 5 muestras para crearse, frenando divisiones basadas en ruido.
- `gamma = 1`: umbral mínimo de reducción de pérdida para realizar una nueva partición; actúa como poda preventiva.

```
Python
best_params = study.best_params
best_params['max_depth'] = min(best_params.get('max_depth', 6), 5)
best_params['min_child_weight'] = 5
best_params['gamma'] = 1

best_model = XGBClassifier(**best_params, random_state=42)
best_model.fit(X_train, y_train)

train_score = best_model.score(X_train, y_train)
test_score = best_model.score(X_test, y_test)
print(f"Train: {train_score:.4f}")
print(f"Test: {test_score:.4f}")
print(f"Diferencia: {abs(train_score - test_score):.4f}")
```

Output:

```
Train: 0.8787
Test: 0.8696
Diferencia: 0.0092
```

Excelente resultado: la diferencia se redujo de 0.0364 a solo 0.0092, confirmando que la regularización funcionó correctamente y el modelo generaliza bien.

FASE 15 — Validación Cruzada Estratificada

La validación cruzada evalúa el modelo en múltiples particiones del dataset, proporcionando una estimación más robusta del rendimiento que una sola división Train-Test. StratifiedKFold garantiza que la proporción de clases se mantenga constante en cada fold.

Con 5 folds: en cada iteración, el modelo se entrena con 4/5 de los datos y se valida con 1/5, rotando hasta cubrir todo el dataset.

```
Python
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(best_model, X, y, cv=skf, scoring='f1')

print(f"F1 por fold: {scores}")
print(f"F1 media: {scores.mean():.4f}")
print(f"Desviación estándar: {scores.std():.4f}")
```

Output:

```
F1 por fold: [0.91089109 0.88669951 0.85446009 0.90821256 0.86384977]
F1 media: 0.8848
Desviación estándar: 0.0228
```

Un F1 medio de 0.8848 con desviación de 0.0228 confirma que el modelo es altamente estable. Una desviación pequeña significa que el rendimiento no depende de qué datos caigan en el fold de validación.

FASE 16 — Balanceo de Clases con SMOTE

Aunque el desbalance original (55% enfermos vs 44% sanos) era moderado, SMOTE permite al modelo aprender con la misma importancia estadística ambas clases. El algoritmo genera muestras sintéticas de la clase minoritaria interpolando entre vecinos cercanos en el espacio de características.

SMOTE se aplica exclusivamente sobre el conjunto de entrenamiento (X_{train} , y_{train}). El conjunto de prueba (X_{test}) conserva la distribución real para una evaluación honesta.

```
Python
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

print(f"Antes - Sano: {y_train.value_counts()[0]}, Enfermo: {y_train.value_counts()[1]}")
```



```
print(f"Después - Sano: {y_train_sm.value_counts()[0]}, Enfermo: {y_train_sm.value_counts()[1]}")
```

Output:

```
Antes - Sano: 333, Enfermo: 401
Después - Sano: 401, Enfermo: 401
```

SMOTE generó 68 registros sintéticos de pacientes sanos para igualar las clases. Ahora el modelo aprenderá con 401 ejemplos de cada clase.

FASE 17 — Análisis de Umbrales de Decisión

Por defecto, un clasificador binario etiqueta como 'Enfermo' a cualquier paciente cuya probabilidad estimada supere 0.5 (50%). En cardiología, este umbral puede ser demasiado conservador: si un paciente tiene 45% de probabilidad de estar enfermo, no debería irse a casa sin más pruebas.

Se evalúan 4 umbrales diferentes (0.5, 0.4, 0.3, 0.2) para analizar el trade-off entre Recall (sensibilidad) y F1-Score:

```
Python
y_prob_sm = best_model_sm.predict_proba(X_test)[:, 1]

for umbral in [0.5, 0.4, 0.3, 0.2]:
    y_pred_t = (y_prob_sm >= umbral).astype(int)
    from sklearn.metrics import f1_score, recall_score
    recall = recall_score(y_test, y_pred_t)
    f1 = f1_score(y_test, y_pred_t)
    print(f"Umbral {umbral} → Recall Enfermo: {recall:.4f} | F1: {f1:.4f}")
```

Output:

```
Umbral 0.5 → Recall Enfermo: 0.8505 | F1: 0.8792
Umbral 0.4 → Recall Enfermo: 0.8692 | F1: 0.8774
Umbral 0.3 → Recall Enfermo: 0.9159 | F1: 0.8829
Umbral 0.2 → Recall Enfermo: 0.9439 | F1: 0.8745
```

Se selecciona 0.4 como umbral final: aumenta el Recall de 0.85 a 0.87 manteniendo un F1 competitivo. Bajar más (0.3, 0.2) incrementa el Recall pero también las falsas alarmas. El criterio clínico prioriza no dejar escapar enfermos.

FASE 18 — Segunda Optimización con Optuna (Maximizar Recall)

La primera optimización maximizó el F1-Score, que equilibra precisión y recall. Para el objetivo médico del proyecto, se realiza una segunda búsqueda donde Optuna optimiza directamente el Recall sobre datos balanceados con SMOTE. Esto obliga al modelo a priorizar la detección de enfermos por encima de todo.

Se restringe max_depth a un máximo de 5 y se fijan gamma=1 y min_child_weight=5 como restricciones de regularización obligatorias.

```

Python
def objective_recall(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 3, 5),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3),
        'subsample': trial.suggest_float('subsample', 0.5, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'reg_alpha': trial.suggest_float('reg_alpha', 0, 10),
        'reg_lambda': trial.suggest_float('reg_lambda', 0, 10),
        'min_child_weight': 5,
        'gamma': 1,
        'random_state': 42
    }
    model = XGBClassifier(**params)
    score = cross_val_score(model, X_train_sm, y_train_sm, cv=5,
scoring='recall').mean()
    return score

study_recall = optuna.create_study(direction='maximize')
study_recall.optimize(
    objective_recall,
    n_trials=200,
    callbacks=[lambda study, trial: study.stop()
                if study.best_trial.number + 20 < trial.number else None]
)
print("Mejores hiperparámetros:", study_recall.best_params)

```

Output:

```

Mejores hiperparámetros: {
  'n_estimators': 369,
  'max_depth': 3,
  'learning_rate': 0.2229,
  'subsample': 0.8813,
  'colsample_bytree': 0.8994,
  'reg_alpha': 1.2819,
  'reg_lambda': 7.7542
}

```

FASE 19 — Construcción del Modelo Maestro Final

Se construye el modelo definitivo combinando todos los elementos desarrollados:

- Hiperparámetros optimizados por Optuna para maximizar Recall.
- Regularización manual: min_child_weight=5 y gamma=1.
- Entrenado sobre datos balanceados con SMOTE.
- Umbral de decisión ajustado a 0.4.

```

Python
best_model_recall = XGBClassifier(
    **study_recall.best_params,
    min_child_weight=5,
    gamma=1,
    random_state=42
)
best_model_recall.fit(X_train_sm, y_train_sm)

```

```

y_prob_recall = best_model_recall.predict_proba(X_test)[:, 1]
y_pred_recall = (y_prob_recall >= 0.4).astype(int)

print(classification_report(y_test, y_pred_recall, target_names=['Sano',
'Enfermo']))

```

Output:

	precision	recall	f1-score	support
Sano	0.85	0.86	0.85	77
Enfermo	0.90	0.89	0.89	107
accuracy			0.88	184

El modelo maestro alcanza Recall=0.89 para enfermos y Accuracy=0.88, mejorando significativamente el modelo base.

Verificación de sobreajuste del modelo maestro

```

Python
train_score = best_model_recall.score(X_train_sm, y_train_sm)
test_score = best_model_recall.score(X_test, y_test)
print(f"Train: {train_score:.4f} | Test: {test_score:.4f} | Diferencia:
{abs(train_score-test_score):.4f}")

```

Output:

Train: 0.9115 | Test: 0.8696 | Diferencia: 0.0419

Una diferencia de 0.0419 es aceptable y confirma que el modelo no está memorizando los datos de entrenamiento.

FASE 20 — Evaluación Final y Exportación del Modelo

Paso 20.1 — Matriz de confusión del modelo final

```

Python
cm = confusion_matrix(y_test, y_pred_recall)
ConfusionMatrixDisplay(cm, display_labels=['Sano', 'Enfermo']).plot(cmap='Blues')
plt.title("Matriz de confusión - Modelo Final")
plt.show()

```

Paso 20.2 — Curva ROC del modelo final

```

Python
RocCurveDisplay.from_estimator(best_model_recall, X_test, y_test)
plt.title("Curva ROC - Modelo Final")
plt.show()

```

Paso 20.3 — Importancia de variables del modelo final

```
Python
plot_importance(best_model_recall, max_num_features=10)
plt.title("Variables más importantes - Modelo Final")
plt.show()
```

Paso 20.4 — Curvas de aprendizaje del modelo final

```
Python
train_sizes, train_scores, val_scores = learning_curve(
    best_model_recall, X, y, cv=skf, scoring='accuracy'
)
plt.plot(train_sizes, train_scores.mean(axis=1), label='Train')
plt.plot(train_sizes, val_scores.mean(axis=1), label='Validación')
plt.title("Curvas de aprendizaje - Modelo Final")
plt.legend()
plt.show()
```

Paso 20.5 — Matriz de confusión normalizada

La versión normalizada muestra porcentajes en lugar de valores absolutos, facilitando la interpretación clínica del rendimiento por clase.

```
Python
cm = confusion_matrix(y_test, y_pred_recall, normalize='true')
ConfusionMatrixDisplay(cm, display_labels=['Sano', 'Enfermo']).plot(
    cmap='Blues', values_format='.0%'
)
plt.title("Matriz de confusión Normalizada - Modelo Final")
plt.show()
```

Paso 20.6 — Curva ROC con AUC calculado manualmente

```
Python
fpr, tpr, _ = roc_curve(y_test, y_prob_recall)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'AUC = {roc_auc:.2f}')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos (Recall)')
plt.title('Curva ROC - Modelo Final')
plt.legend()
plt.show()
```

Paso 20.7 — Guardado del modelo (joblib)

El modelo maestro se serializa en un archivo .pkl mediante joblib. Este archivo encapsula el modelo entrenado completo: pesos, estructura de árboles e hiperparámetros. Puede cargarse en cualquier entorno Python sin necesidad de reentrenar.

```
Python
joblib.dump(best_model_recall, '../data/model_R.pkl')
```

Output:

```
[ '../data/model_R.pkl' ]
```

Para usar el modelo en producción: `modelo = joblib.load('../data/model_R.pkl')` → ya está listo para hacer predicciones.

Paso 20.8 — Exportación del dataset procesado

El dataset con todas las transformaciones aplicadas (imputación, encoding, columnas binarias) se guarda en CSV para ser reutilizado sin necesidad de ejecutar de nuevo el preprocesamiento.

```
Python
data.to_csv('heart_processed_R.csv', index=False)
```

Resumen de Resultados Finales

Métrica	Valor
Accuracy global	88%
Precisión (Enfermo)	90%
Recall (Enfermo)	89%
F1-Score (Enfermo)	89%
F1 media validación cruzada (5 folds)	0.8848
Desviación estándar validación cruzada	0.0228
Diferencia Train-Test (overfitting)	0.0419
Umbral de decisión clínico	0.4
Dataset de entrenamiento (con SMOTE)	802 pacientes (401 + 401)
Dataset de prueba (real)	184 pacientes

El flujo completo del notebook sigue 20 fases que van desde la carga cruda de datos hasta la exportación de un modelo de producción, pasando por EDA detallado, limpieza clínica, codificación de variables, análisis visual, entrenamiento base, optimización bayesiana doble (F1 y Recall), balanceo de clases con SMOTE, análisis de umbrales y validación estadística exhaustiva. El resultado es un sistema de diagnóstico cardíaco con un 89% de sensibilidad, listo para integrarse en entornos de apoyo al diagnóstico médico en tiempo real.